

CLASS **LinearRegression** Gradient Descent · 1-D Linear Model · From Scratch

Model: $\hat{y} = W \cdot X + b$ **Minimize:** $J(W, b) = \sum_i (\hat{y}_i - y_i)^2$ **Update:** $\theta \leftarrow \theta - \alpha \frac{\partial J}{\partial \theta}$

`__init__` Constructor

Purpose: Initialize model parameters W (slope) and b (intercept) to random values for subsequent optimization.

```
FUNCTION __init__(self):
    # Fix random seed for reproducibility
    SET random_seed <- 413

    # Initialize parameters randomly in [0, 1)
    SET self.W <- random_float()      # slope
    SET self.b <- random_float()      # intercept
```

Note: A fixed seed ensures that every run starts from the same (W_0, b_0) , making experiments reproducible. Both parameters are scalars because this is a one-dimensional model.

`predict` Prediction function

Purpose: Apply the current linear model to input data X .

```
FUNCTION predict(self, X):
    # Evaluate the linear hypothesis
    RETURN self.W * X + self.b      # y_hat = W*X + b
```

Note: X may be a scalar or a vector. When X is an array the multiplication is element-wise, so $\hat{y}$ has the same shape as X .

`cost` Cost function

Purpose: Measure how far predictions are from the true targets using the *Sum of Squared Residuals* (SSR):

$$J(W, b) = \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

```
FUNCTION cost(self, X, y):
    COMPUTE delta <- predict(X) - y      # residuals: (y_hat - y)
    RETURN SUM( delta ** 2 )              # J = sum of squared residuals
```

Note: $J \geq 0$ always. SSR is chosen over absolute error because it is differentiable everywhere, a requirement for gradient descent.

grad_cost Gradient of the cost function

Purpose: Compute the partial derivatives of J with respect to each parameter. These point in the direction of steepest ascent, so we step in the *opposite* direction during training.

$$\frac{\partial J}{\partial W} = 2 \sum_i (\hat{y}_i - y_i) x_i \qquad \frac{\partial J}{\partial b} = 2 \sum_i (\hat{y}_i - y_i)$$

```
FUNCTION grad_cost(self, X, y):
    COMPUTE delta <- predict(X) - y           # residuals vector

    # Partial derivatives via the chain rule applied to SSR
    COMPUTE dJ_dW <- 2 * SUM( delta * X )     # partial dJ/dW
    COMPUTE dJ_db <- 2 * SUM( delta )         # partial dJ/db

    RETURN dJ_dW, dJ_db
```

Derivation sketch: $J = \sum_i (Wx_i + b - y_i)^2$. Chain rule gives $\frac{\partial J}{\partial W} = \sum_i 2(Wx_i + b - y_i) \cdot x_i = 2 \sum_i \delta_i x_i$. The factor 2 is exact and in practice is absorbed into the learning rate α .

fit Optimization via gradient descent

Purpose: Iteratively update W and b to minimize J . Training halts when convergence is detected or the iteration budget is exhausted.

```
FUNCTION fit(self, X, y):
    SET alpha <- 0.0001           # learning rate (step size)
    SET tol <- 1e-13              # convergence tolerance
    SET history <- [ cost(X, y) ] # track J at every epoch
    SET epoch <- 0
    SET epsilon <- 1              # initial cost change (> tol)

    PRINT "Initial cost = ", history[0]

    # -- Gradient descent loop -----
    WHILE epsilon > tol AND epoch < 250000:

        # 1. Compute gradients at current (W, b)
        COMPUTE dJ_dW, dJ_db <- grad_cost(X, y)

        # 2. Move W and b opposite to the gradient
        UPDATE self.W <- self.W - alpha * dJ_dW
        UPDATE self.b <- self.b - alpha * dJ_db

        # 3. Record new cost
        APPEND history <- cost(X, y)

        # 4. Convergence check: change in J between consecutive epochs
        COMPUTE epsilon <- | history[epoch] - history[epoch + 1] |

        INCREMENT epoch <- epoch + 1

    # -- End loop -----

    PRINT "Final cost = ", history[-1]
```

```
PRINT "Total epochs = ", epoch

RETURN self.W, self.b
```

Stopping criteria — two independent conditions:

- **Convergence:** $\varepsilon = |J^{(t)} - J^{(t+1)}| < \text{tol}$ — the cost has stopped improving meaningfully.
- **Budget:** $\text{epoch} \geq 250,000$ — safety cap against very slow convergence.

The `history` list enables post-training plots of the *learning curve* (loss vs. epoch).

`score` R^2 coefficient of determination

Purpose: Evaluate goodness of fit after training. $R^2 \in (-\infty, 1]$, where $R^2 = 1$ indicates a perfect fit.

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

```
FUNCTION score(self, X, y):
  SET n <- LENGTH(y)
  COMPUTE y_mean <- (1/n) * SUM(y)           # mean of targets

  COMPUTE SS_res <- SUM( (predict(X) - y)**2 ) # residual SS
  COMPUTE SS_tot <- SUM( (y - y_mean)**2 )     # total SS

  RETURN 1 - SS_res / SS_tot                 # R^2
```

Interpretation:

- $R^2 = 1$ — model explains all variance in y .
- $R^2 = 0$ — model is no better than always predicting $\bar{y}$.
- $R^2 < 0$ — model performs worse than the mean baseline.

SS_{tot} captures the intrinsic variance of the targets; SS_{res} is the unexplained variance after fitting.